

Lab: Analyzing TCP and UDP Traffic with Wireshark

Estimated time: **30** minutes

Objectives

After completing this lab you'll be able to analyze TCP and UDP network traffic. You'll also be able to demonstrate the skills needed to install, configure, and use an open-source network analysis tool, Wireshark, to perform these tasks.

Tools Needed

- Wireshark, a no-charge open-source network analysis tool, installed on your computer.

Note: Wireshark is available for MacOS as well as for Microsoft Windows. This lab uses the Windows version of this application.

Instructions for Downloading and Installing Wireshark

Before you begin your network analysis lab tasks, you'll need to install and configure Wireshark on your local computer system. Follow these steps to get started:

Downloading Wireshark

1. To download Wireshark, visit the official Wireshark website, <https://www.wireshark.org/download.html>
2. Select the Windows installer to download the latest stable version of Wireshark.

Installing Wireshark

1. Locate the installer on your machine.
2. Click to install and follow the on-screen instructions, accepting the default options.
3. During the installation, you might be prompted to install WinPcap or Npcap (necessary for capturing network traffic). If so, choose **Yes** and complete the installation.

Important: If running Wireshark in a virtual environment, do not reboot or restart then environment.

Configuring Wireshark for Network Analysis

If Wireshark is not already running, right click on Wireshark icon and select **Run as Administrator**.

Set Up Your Network Interfaces

1. Upon launching Wireshark, the application displays a list of available network interfaces including wireless and Ethernet networks.
2. Select the interface you want to monitor. Typically, this interface will be your active network connection such as your wireless and Ethernet network.

Configure Capture Filters (Optional)

Before starting a capture, you can set up filters to focus on specific traffic such as tcp, udp, or specific ports.

To add filters, locate the **Capture Filter** field on the main screen, or after starting the capture, you can enter filters in the **Display Filter** bar.

Testing the Installation

1. Start a new capture by selecting your network interface and clicking the blue shark fin icon (Start Capture).
2. Open a web browser and visit any website to generate traffic.
3. Stop the capture after a few seconds by clicking the red square icon (Stop Capture).
4. You should see the captured packets displayed in Wireshark, confirming that the installation is working correctly.

Saving Your Capture (Optional)

After stopping the capture, you can save the capture file for later analysis by selecting **File > Save As...** and selecting a location on your system.

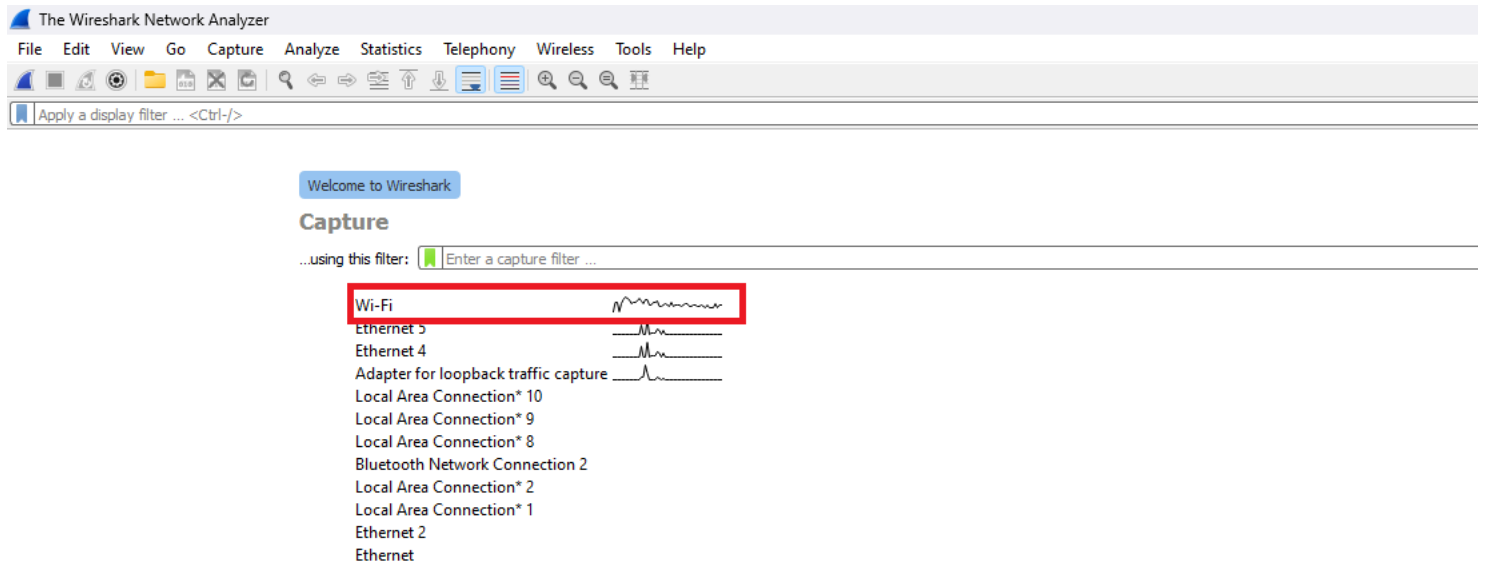
Now that you installed and configure Wireshark, you're ready to begin the lab exercises!

Tasks

Capture TCP Traffic

Steps:

1. Rick-click the Wireshark icon and select **Run as Administrator** to start a new capture session.



2. In the filter bar, enter `tcp` and select **Enter** to filter TCP packets.

Capturing from Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.241.84? Tell 172.16.223.136
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.240.122? Tell 172.16.239.80
1	0.000000	172.16.205.231	224.0.0.251	MDNS	161	Standard query 0x0000 PTR _companion-link._tcp.local, "QU" question PTR _rdlin
1	0.000000	172.16.205.231	224.0.0.251	MDNS	141	Standard query 0x0000 PTR _companion-link._tcp.local, "QU" question PTR _rdlin
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.227.47? Tell 172.16.209.151
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.223.136? Tell 172.16.235.98
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.218.206? Tell 172.16.209.133
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.227.47? Tell 172.16.234.183
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.240.122? Tell 172.16.225.132
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.241.84? Tell 172.16.220.247
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.235.45? Tell 172.16.206.174
1	0.000000	172.16.205.231	224.0.0.251	SSDP	217	M-SEARCH * HTTP/1.1
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.227.47? Tell 172.16.232.215
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.227.30? Tell 172.16.232.215
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.227.47? Tell 172.16.230.153
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.227.183? Tell 172.16.230.153
1	0.000000	172.16.205.231	224.0.0.251	MDNS	146	Standard query 0x0000 ANY Android-32.local, "QM" question ANY Android-32.local
1	0.000000	172.16.205.231	224.0.0.251	MDNS	126	Standard query 0x0000 ANY Android-32.local, "QM" question ANY Android-32.local
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.230.100? Tell 172.16.0.1
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.234.49? Tell 172.16.236.108
1	0.000000	172.16.205.231	224.0.0.251	MDNS	607	Standard query response 0x0000 A, cache flush 172.16.11.100 SRV, cache flush 0
1	0.000000	172.16.205.231	224.0.0.251	ARP	42	Who has 172.16.233.89? Tell 172.16.218.13
1	0.000000	172.16.205.231	224.0.0.251	IGMPv3	54	Membership Report / Join group 239.255.255.250 for any sources
1	0.000000	172.16.205.231	224.0.0.251	ARP	60	Who has 172.16.221.112? (ARP Probe)
1	0.000000	172.16.205.231	224.0.0.251	ICMPv6	86	Neighbor Advertisement fe80::b80b:c01e:c9b6:2875 (ovr) is at c8:8a:9a:e1:04:15

> Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{FCSA2527-7DB6-4BCD-B05B-8E59DD34FB3A}, i

> Ethernet II, Src: 9e:8a:73:41:ed:f3 (9e:8a:73:41:ed:f3), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

> Address Resolution Protocol (request)

0000 ff ff ff ff ff ff
0010 08 00 06 04 00 00
0020 00 00 00 00 00 00
0030 00 00 00 00 00 00

3. Open a web browser and visit a website, such as <https://www.ibm.com/in-en> to generate TCP traffic.

4. Stop the capture after a few seconds.

Capture TCP data

1. Examine the captured packets. Identify packets involved in the TCP three-way handshake including SYN, SYN-ACK, and ACK.

*Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp

No.	Time	Source	Destination	Protocol	Length	Info
1607...	296.374124	172.16.218.13	172.16.205.231	TCP	1514	[TCP ACKed unseen segment] 7680 → 53667 [ACK] Seq=1237649 Ack=549 Win=1049088 Len=1460
1607...	296.374211	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53667 [ACK] Seq=1239109 Ack=549 Win=1049088 Len=1460
1607...	296.374302	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53667 [ACK] Seq=1240569 Ack=549 Win=1049088 Len=1460
1607...	296.374302	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53667 [ACK] Seq=1242029 Ack=549 Win=1049088 Len=1460
1607...	296.374302	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53667 [ACK] Seq=1243489 Ack=549 Win=1049088 Len=1460
1607...	296.374302	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53667 [PSH, ACK] Seq=1244949 Ack=549 Win=1049088 Len=1460
1607...	296.374302	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53667 [ACK] Seq=1246409 Ack=549 Win=1049088 Len=1460
1607...	296.374302	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53667 [ACK] Seq=1247869 Ack=549 Win=1049088 Len=1460
1607...	296.374302	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53667 [ACK] Seq=1249329 Ack=549 Win=1049088 Len=1460
1607...	296.374302	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53667 [ACK] Seq=1250789 Ack=549 Win=1049088 Len=1460
1607...	296.374394	172.16.218.13	172.16.205.231	TCP	1514	[TCP Retransmission] 7680 → 53649 [PSH, ACK] Seq=1728018 Ack=550 Win=524800 Len=1460
1607...	296.374394	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53649 [ACK] Seq=1733858 Ack=550 Win=524800 Len=1460
1607...	296.374394	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53649 [ACK] Seq=1735318 Ack=550 Win=524800 Len=1460
1607...	296.374394	172.16.218.13	172.16.205.231	TCP	544	7680 → 53649 [PSH, ACK] Seq=1736778 Ack=550 Win=524800 Len=490
1607...	296.384868	172.16.205.231	172.16.218.13	TCP	66	53649 → 7680 [ACK] Seq=550 Ack=1725098 Win=525568 Len=0 SLE=1729478 SRE=1732398
1607...	296.384868	172.16.205.231	172.16.218.13	TCP	66	53649 → 7680 [ACK] Seq=550 Ack=1726558 Win=525568 Len=0 SLE=1729478 SRE=1732398
1607...	296.384868	172.16.205.231	172.16.218.13	TCP	66	53649 → 7680 [ACK] Seq=550 Ack=1728018 Win=525568 Len=0 SLE=1729478 SRE=1732398
1607...	296.384935	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53649 [ACK] Seq=1737268 Ack=550 Win=524800 Len=1460
1607...	296.384935	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53649 [ACK] Seq=1738728 Ack=550 Win=524800 Len=1460
1607...	296.384935	172.16.218.13	172.16.205.231	TCP	1514	7680 → 53649 [ACK] Seq=1740188 Ack=550 Win=524800 Len=1460
1607...	296.391654	16.163.136.108	172.16.218.13	TLSv1.2	1514	Server Hello
1607...	296.391654	16.163.136.108	172.16.218.13	TCP	1514	443 → 54886 [ACK] Seq=1461 Ack=518 Win=28160 Len=1460 [TCP segment of a reassembled
1607...	296.391654	16.163.136.108	172.16.218.13	TCP	1514	443 → 54886 [ACK] Seq=2921 Ack=518 Win=28160 Len=1460 [TCP segment of a reassembled
1607...	296.391654	16.163.136.108	172.16.218.13	TLSv1.2	1084	Certificate, Server Key Exchange, Server Hello Done
1607...	296.391654	16.163.136.108	172.16.218.13	TLSv1.2	968	Application Data

> Frame 222: 55 bytes on wire (440 bits), 55 bytes captured (440 bits) on interface \Device\NPF_{FCSA2527-7DB6-4BCD-B05B-8E59DD34FB3A}, i

> Ethernet II, Src: IntelCor_96:66:e9 (40:ec:99:96:66:e9), Dst: Sophos_d1:41:00 (7c:5a:1c:d1:41:00)

> Internet Protocol Version 4, Src: 172.16.218.13, Dst: 62.67.238.152

> Transmission Control Protocol, Src Port: 54496, Dst Port: 443, Seq: 1, Ack: 1, Len: 1

0000 7c 5a 1c d1 41
0010 00 29 98 65 40
0020 ee 98 d4 e0 01
0030 f7 5b 08 a6 00

2. Examine the TCP header for sequence numbers, acknowledgments, and flags such as SYN and ACK.

Wireshark · Packet 160735 · Wi-Fi

```

> Frame 160735: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface \Device\NPF_{FC5A2527-7DB6-48CD-B05B-8E59DD34FB3A}, id 0
> Ethernet II, Src: IntelCor_96:66:e9 (40:ec:99:96:66:e9), Dst: IntelCor_35:50:7a (48:51:c5:35:50:7a)
> Internet Protocol Version 4, Src: 172.16.218.13, Dst: 172.16.205.231
> Transmission Control Protocol, Src Port: 7680, Dst Port: 53667, Seq: 1239109, Ack: 549, Len: 1460
  Source Port: 7680
  Destination Port: 53667
  [Stream index: 228]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 1460]
  Sequence Number: 1239109 (relative sequence number)
  Sequence Number (raw): 486568610
  [Next Sequence Number: 1240569 (relative sequence number)]
  Acknowledgment Number: 549 (relative ack number)
  Acknowledgment number (raw): 1378563244
  0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 4098
    [Calculated window size: 1049088]
    [Window size scaling factor: 256]

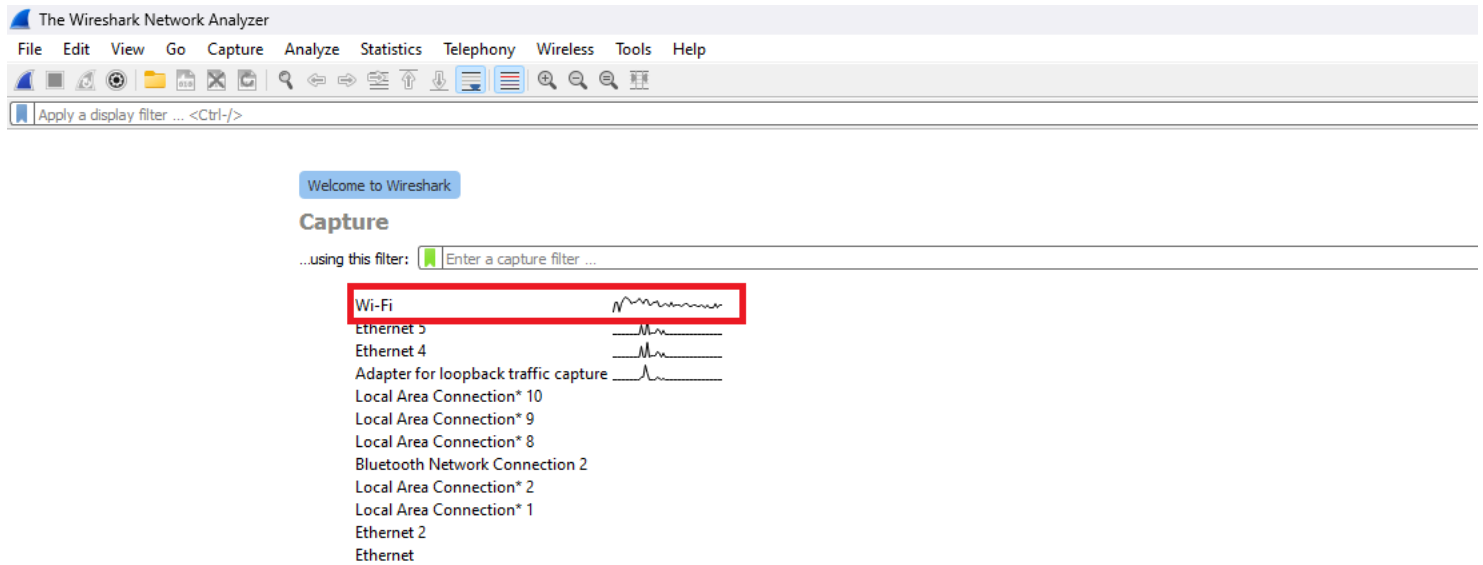
```

0000	48 51 c5 35 50 7a 40 ec 99 96 66 e9 08 00 45 00	HQ:5Pz@...f...E:
0010	05 dc 76 f8 40 00 80 06 7e 0d ac 10 da 0d ac 10	...v:@...~.....
0020	cd e7 1e 00 d1 a3 1d 00 72 a2 52 2b 34 ac 50 10	r.R+4.P.
0030	10 02 b6 79 00 00 be 81 ac 55 f4 ad 27 7a 85 f9	...y...U...z...
0040	88 b2 ca fe 90 3c 32 a8 6a 41 9d 6a 86 bd bf 03	<2..jA.j....
0050	a9 d0 ab 52 26 02 86 1b 36 ee c8 3e ea b0 b9 b1	...R&...6...>....
0060	6e 24 d8 1d f4 ae 4e 60 8a c1 7e 52 70 f5 48 22	n\$...N`...~Rp.H"
0070	f0 9c 7c cc 14 83 3d 64 f2 f9 82 a5 df 64 1c f1	=dd...
0080	eb d7 b9 b7 38 7e d1 f3 57 6e 56 cf 9b 31 e7 75	...8...WnV...1.u
0090	fe 68 4c d5 a4 4a ef 77 67 81 a9 6a 8d 79 c8 2c	..hL...J.w g...j.y,
00a0	55 e3 af 76 9b 58 a9 e3 f9 c8 58 e5 0b 0f d0 de	U...v.X...X.....
00b0	ad 8e bc f3 ad 7d 3a 47 53 4f ff f4 69 63 02 d6	}:G SO...ic...
00c0	56 a2 cf f0 68 9d ec 33 d4 ce d0 b7 23 be 55 eb	V...h...3 ...#..U..
00d0	dc 61 2d cd f3 89 f3 bc 7e 7b c4 bb db 11 2f a6	..a.....~{..../. ..
00e0	73 53 d2 02 93 52 a6 a4 34 f9 3e 35 59 04 93 f4	sS...R...4>5Y...
00f0	7e 6a 02 70 86 8b 7e 5d 31 95 42 e0 fb 9e 09 42	~j.p...~] 1.B...B
0100	c4 ac 9c 29 29 8f 49 eb aa a9 42 7c 77 63 7a 65	...))..I...B wcze
0110	02 e3 0a db ba 36 b5 85 f2 f0 4c c3 90 94 c5 ed	6...L.....
0120	be df 32 95 42 26 a1 d4 32 d8 2c 5f b6 20 c3 b4	...2.B&...2..._... ..
0130	63 02 03 19 85 8c 29 29 8b 49 fd 3b 53 5b 68 2f	c.....))..I.;S[h/
0140	3c bf 37 cd 17 9e 05 77 0f 86 a4 7d 10 89 3f 8e	<.7...w ...>..?..
0150	35 a8 d9 17 63 06 e6 97 ba 33 9c 62 ec e6 bc f3	5...c...3.b....
0160	be 50 5e 34 c6 47 05 c9 c8 8b bb b3 77 99 80 8a	..P^4.G...w...
0170	4d 30 f6 72 af e7 f7 32 1a d4 da e5 47 85 09 13	M0.r...2 ...G...
0180	44 b7 d6 03 2c 08 b3 3d 07 a8 a4 ec e1 4f 07 53	D...,,=O.S

☒ Show packet bytes

Capture UDP Traffic

1. In Wireshark, clear the previous capture and start a new session.



2. In the filter bar, enter `udp` and select **Enter** to filter UDP packets.

Capturing from Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

udp

No.	Time	Source	Destination	Protocol	Length	Info
35805	43.862440	IntelCor_a3:38:5b	Broadcast	ARP	60	Who has 172.16.217.170? Tell 172.16.214.103
35806	43.862440	e2:f7:d6:dc:72:bd	Broadcast	ARP	60	Who has 172.16.235.23? Tell 172.16.223.97
35807	43.862440	172.16.11.110	224.0.0.251	MDNS	102	Standard query response 0x0000 A, cache flush 172.16.11.110 NSEC, cache flush
35808	43.862440	172.16.11.115	224.0.0.251	MDNS	190	Standard query response 0x0000 A, cache flush 172.16.11.115 SRV, cache flush
35809	43.862440	172.16.233.89	224.0.0.251	MDNS	404	Standard query response 0x0000 PTR iMacLab_googlecast._tcp.local PTR iMacLab.
35810	43.862440	fe80::bfa:b8ff:fed...	ff02::fb	MDNS	424	Standard query response 0x0000 PTR iMacLab_googlecast._tcp.local PTR iMacLab.
35811	43.862440	ArubaaHe_c3:93:50	Broadcast	IAP	313	Aruba Instant AP
35812	43.862440	ArubaaHe_c3:93:50	Broadcast	IAP	313	Aruba Instant AP
35813	43.862440	fe80::4305:82ca:961...	ff02::c	UDP	1154	60974 → 3702 Len=1092
35814	43.862440	172.16.205.188	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.1
35815	43.862440	a6:f4:79:59:cb:35	Broadcast	ARP	60	Who has 172.16.179.77? Tell 172.16.230.184
35816	43.862440	6e:a0:3e:66:9a:98	Broadcast	ARP	60	Who has 172.16.176.50? Tell 172.16.238.177
35817	43.862440	a6:f4:79:59:cb:35	Broadcast	ARP	60	Who has 172.16.235.23? Tell 172.16.230.184
35818	43.862440	XiaomiCo_ff:e0:ec	Broadcast	ARP	60	Who has 172.16.234.49? Tell 172.16.212.109
35819	43.862440	ae:13:7c:1f:9c:0f	Broadcast	ARP	60	Who has 172.16.244.1? Tell 172.16.213.85
35820	43.862440	52:e9:c6:af:04:6d	Broadcast	ARP	60	Who has 172.16.237.230? Tell 172.16.220.236
35821	43.862440	IntelCor_a3:38:5b	Broadcast	ARP	60	Who has 172.16.217.171? Tell 172.16.214.103
35822	43.862440	a6:f4:79:59:cb:35	Broadcast	ARP	60	Who has 172.16.176.50? Tell 172.16.230.184
35823	43.862440	XiaomiCo_ff:e0:ec	Broadcast	ARP	60	Who has 172.16.230.240? Tell 172.16.212.109
35824	43.862440	52:e9:c6:af:04:6d	Broadcast	ARP	60	Who has 172.16.226.24? Tell 172.16.220.236
35825	43.862440	IntelCor_a3:38:5b	Broadcast	ARP	60	Who has 172.16.217.172? Tell 172.16.214.103
35826	43.862440	66:e3:24:40:04:f2	Broadcast	ARP	60	Who has 172.16.241.116? Tell 172.16.223.253
35827	43.862440	ae:13:7c:1f:9c:0f	Broadcast	ARP	60	Who has 172.16.230.240? Tell 172.16.213.85
35828	43.862440	XiaomiCo_ff:e0:ec	Broadcast	ARP	60	Who has 172.16.233.18? Tell 172.16.212.109
35829	43.862440	5e:a2:4e:7c:8b:4e	Broadcast	ARP	60	Who has 172.16.240.122? Tell 172.16.206.174

> Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{FCSA2527-7DB6-4B0D-B05B-8E59DD34FB3A}, i

> Ethernet II, Src: 26:dd:32:c8:8d:d3 (26:dd:32:c8:8d:d3), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

> Address Resolution Protocol (request)

3. Use an application that generates UDP traffic, such as a DNS query or a video streaming service. Allow the service to run for a few seconds.

*Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

udp

No.	Time	Source	Destination	Protocol	Length	Info
14289	19.608999	fe80::14c5:fba2:a51...	ff02::fb	MDNS	981	Standard query 0x0000 PTR _airport._tcp.local, "QU" question PTR _rdlink._tcp.
14290	19.608999	172.16.225.26	224.0.0.251	MDNS	961	Standard query 0x0000 PTR _airport._tcp.local, "QU" question PTR _rdlink._tcp.
14291	19.619302	172.16.225.26	224.0.0.251	MDNS	892	Standard query 0x0000 PTR _airport._tcp.local, "QU" question PTR BytelloShare4
14293	19.619302	fe80::14c5:fba2:a51...	ff02::fb	MDNS	912	Standard query 0x0000 PTR _airport._tcp.local, "QU" question PTR BytelloShare4
14294	19.619302	172.16.225.26	224.0.0.251	MDNS	1128	Standard query 0x0000 PTR _airport._tcp.local, "QU" question PTR BytelloShare4751
14295	19.619302	fe80::14c5:fba2:a51...	ff02::fb	MDNS	1148	Standard query 0x0000 PTR _airport._tcp.local, "QU" question PTR BytelloShare4751
14297	19.619302	172.16.240.154	224.0.0.251	MDNS	124	Standard query 0x0000 A KM3CB61E.local, "QM" question SRV KONICAMINOLTA-bizhub
14331	19.700323	172.16.229.142	224.0.0.251	MDNS	152	Standard query 0x0003 PTR _9E5E7C8F47989526C9BCD95D24084F6F0B27C5ED._sub._goo
14340	19.700323	0.0.0.0	255.255.255.255	DHCP	356	DHCP Request - Transaction ID 0xb1160731
14343	19.700323	172.16.230.213	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.1
14364	19.707142	172.16.207.148	224.0.0.251	MDNS	224	Standard query 0x0000 ANY {"nm": "KING'S COURT", "as": "[8194]", "ip": "148"}.
14365	19.707142	fe80::a873:b35:c4a7...	ff02::fb	MDNS	244	Standard query 0x0000 ANY {"nm": "KING'S COURT", "as": "[8194]", "ip": "148"}.
14397	19.803084	172.16.11.110	224.0.0.251	MDNS	359	Standard query response 0x0000 PTR HP LaserJet Pro MFP M226dw (B2C7B1)._ipp._t
14398	19.803084	fe80::ead8:d1ff:feb...	ff02::fb	MDNS	379	Standard query response 0x0000 PTR HP LaserJet Pro MFP M226dw (B2C7B1)._ipp._t
14431	19.813679	172.16.241.170	224.0.0.251	MDNS	142	Standard query response 0x0000 PTR RVU11 iMac._airport._tcp.local PTR 3C7D0A1D
14435	19.813679	fe80::1c8f:9f03:db1...	ff02::fb	MDNS	162	Standard query response 0x0000 PTR RVU11 iMac._airport._tcp.local PTR 3C7D0A1D
14438	19.813679	172.16.242.52	224.0.0.251	MDNS	144	Standard query response 0x0000 PTR RV's iMac._airport._tcp.local PTR 3C7D0A17B
14439	19.813679	fe80::1009:bea9:38d...	ff02::fb	MDNS	164	Standard query response 0x0000 PTR RV's iMac._airport._tcp.local PTR 3C7D0A17B
14441	19.813679	172.16.224.82	224.0.0.251	MDNS	365	Standard query response 0x0000 PTR, cache flush LAPTOP-0PB2Q5AQ._dosvc._tcp.lo
14442	19.814357	fe80::8292:9403:591...	ff02::fb	MDNS	385	Standard query response 0x0000 PTR, cache flush LAPTOP-0PB2Q5AQ._dosvc._tcp.lo
14443	19.814357	172.16.224.82	224.0.0.251	MDNS	301	Standard query response 0x0000 SRV, cache flush 0 0 7680 LAPTOP-0PB2Q5AQ.local
14444	19.814357	fe80::8292:9403:591...	ff02::fb	MDNS	321	Standard query response 0x0000 SRV, cache flush 0 0 7680 LAPTOP-0PB2Q5AQ.local
14446	19.814357	172.16.233.89	224.0.0.251	MDNS	375	Standard query response 0x0000 PTR iMacLab_googlecast._tcp.local TXT, cache f
14447	19.814357	fe80::bfa:b8ff:fed...	ff02::fb	MDNS	395	Standard query response 0x0000 PTR iMacLab_googlecast._tcp.local TXT, cache f
14466	19.835264	172.16.225.126	224.0.0.251	MDNS	168	Standard query 0x0003 PTR _9E5E7C8F47989526C9BCD95D24084F6F0B27C5ED._sub._goo

4. Stop the capture after a few seconds.

Analyzing the data

1. Examine the captured packets and identify the source and destination ports.

Wireshark · Packet 14435 · Wi-Fi

```

> Frame 14435: 162 bytes on wire (1296 bits), 162 bytes captured (1296 bits) on interface \Device\NPF_{FC5A2527-7DB6-4BCD-B05B-8E59DD34FB3A}, id 0
> Ethernet II, Src: Apple_9d:ff:05 (5c:52:30:9d:ff:05), Dst: IPv6mcast_fb (33:33:00:00:00:fb)
> Internet Protocol Version 6, Src: fe80::1c8f:9f03:db15:d9b9, Dst: ff02::fb
User Datagram Protocol, Src Port: 5353, Dst Port: 5353
  Source Port: 5353
  Destination Port: 5353
  Length: 108
  Checksum: 0xd219 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 177]
  > [Timestamps]
  UDP payload (100 bytes)
> Multicast Domain Name System (response)

```

0000	33 33 00 00 00 fb 5c 52 30 9d ff 05 86 dd 60 00	33.....\R 0.....
0010	00 00 00 6c 11 ff fe 80 00 00 00 00 00 00 1c 8f	...l.....
0020	9f 03 db 15 d9 b9 ff 02 00 00 00 00 00 00 00 00	
0030	00 00 00 00 00 fb 14 e9 14 e9 00 6c d2 19 00 00	l.....
0040	84 00 00 00 00 02 00 00 00 00 08 5f 61 69 72 70	_airp
0050	6c 61 79 04 5f 74 63 70 05 6c 6f 63 61 6c 00 00	lay_tcp_local..
0060	0c 00 01 00 00 11 94 00 0d 0a 52 56 55 31 31 20	RVU11
0070	69 4d 61 63 c0 0c 05 5f 72 61 6f 70 c0 15 00 0c	iMac...raop...
0080	00 01 00 00 11 94 00 1a 17 33 43 37 44 30 41 31	3C7D0A1
0090	44 34 32 33 31 40 52 56 55 31 31 20 69 4d 61 63	D4231@RV U11 iMac
00a0	c0 38	8

No.: 14435 · Time: 19.813679 · Source: fe80::1c8f:9f03:db15:d9b9 · Destination: ff02::fb · Protocol: MDNS · Length: 162 · Info: Standard query response 0x0000 PTR RVU11 iMac_airplay_tcp.local PTR 3C7D0A1D4231@RVU11 iMac_raop_tcp.local

☒ Show packet bytes

2. Observe the UDP header for the length and checksum fields.

Wireshark · Packet 14435 · Wi-Fi

```

> Frame 14435: 162 bytes on wire (1296 bits), 162 bytes captured (1296 bits) on interface \Device\NPF_{FC5A2527-7DB6-4BCD-B05B-8E59DD34FB3A}, id 0
> Ethernet II, Src: Apple_9d:ff:05 (5c:52:30:9d:ff:05), Dst: IPv6mcast_fb (33:33:00:00:00:fb)
> Internet Protocol Version 6, Src: fe80::1c8f:9f03:db15:d9b9, Dst: ff02::fb
  User Datagram Protocol, Src Port: 5353, Dst Port: 5353
    Source Port: 5353
    Destination Port: 5353
    Length: 108
    Checksum: 0xd219 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 177]
  > [Timestamps]
    UDP payload (100 bytes)
  > Multicast Domain Name System (response)

```

0000	33 33 00 00 00 fb 5c 52 30 9d ff 05 86 dd 60 00	33.....\R 0.....
0010	00 00 00 6c 11 ff fe 80 00 00 00 00 00 00 1c 8f	...l.....
0020	9f 03 db 15 d9 b9 ff 02 00 00 00 00 00 00 00 00	
0030	00 00 00 00 00 00 fb 14 e9 14 e9 00 6c d2 19 00 00	l....
0040	84 00 00 00 00 02 00 00 00 00 08 5f 61 69 72 70	_airp
0050	6c 61 79 04 5f 74 63 70 05 6c 6f 63 61 6c 00 00	lay_tcp_local..
0060	0c 00 01 00 00 11 94 00 0d 0a 52 56 55 31 31 20	RVU11
0070	69 4d 61 63 c0 0c 05 5f 72 61 6f 70 c0 15 00 0c	iMac...raop...
0080	00 01 00 00 11 94 00 1a 17 33 43 37 44 30 41 31	3C7D0A1
0090	44 34 32 33 31 40 52 56 55 31 31 20 69 4d 61 63	D4231@RV U11 iMac
00a0	c0 38	.8

No.: 14435 · Time: 19.813679 · Source: fe80::1c8f:9f03:db15:d9b9 · Destination: ff02::fb · Protocol: MDNS · Length: 162 · Info: Standard query response 0x0000 PTR RVU11 iMac_airplay_tcp.local PTR 3C7D0A1D4231@RVU11 iMac_raop_tcp.local

☒ Show packet bytes

Compare TCP and UDP Packets

1. Compare a TCP packet to a UDP packet captured in previous tasks.

- View a TCP HEADER

Wireshark · Packet 160735 · Wi-Fi

```

> Frame 160735: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface \Device\NPF_{
> Ethernet II, Src: IntelCor_96:66:e9 (40:ec:99:96:66:e9), Dst: IntelCor_35:50:7a (48:51:c5:35:50:7a)
> Internet Protocol Version 4, Src: 172.16.218.13, Dst: 172.16.205.231
  Transmission Control Protocol, Src Port: 7680, Dst Port: 53667, Seq: 1239109, Ack: 549, Len: 1460
    Source Port: 7680
    Destination Port: 53667
    [Stream index: 228]
    [Conversation completeness: Incomplete, DATA (15)]
    [TCP Segment Len: 1460]
    Sequence Number: 1239109 (relative sequence number)
    Sequence Number (raw): 486568610
    [Next Sequence Number: 1240569 (relative sequence number)]
    Acknowledgment Number: 549 (relative ack number)
    Acknowledgment number (raw): 1378563244
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 4098
    [Calculated window size: 1049088]
    [Window size scaling factor: 256]

```


- View a UDP HEADER

Wireshark · Packet 14435 · Wi-Fi

> Frame 14435: 162 bytes on wire (1296 bits), 162 bytes captured (1296 bits) on interface \Device\NPF_{FC5A2527-7DB6-4BCD-B05B-8E59DD34FB3A}, id 0

> Ethernet II, Src: Apple_9d:ff:05 (5c:52:30:9d:ff:05), Dst: IPv6mcast_fb (33:33:00:00:00:fb)

> Internet Protocol Version 6, Src: fe80::1c8f:9f03:db15:d9b9, Dst: ff02::fb

> User Datagram Protocol, Src Port: 5353, Dst Port: 5353

Source Port: 5353

Destination Port: 5353

Length: 108

Checksum: 0xd219 [unverified]

[Checksum Status: Unverified]

[Stream index: 177]

> [Timestamps]

UDP payload (100 bytes)

> Multicast Domain Name System (response)

2. Note the differences in the headers of TCP and UDP packets.

Analyzing the data

- TCP headers include sequence and acknowledgment numbers and flags (SYN, ACK), whereas UDP headers do not.
- UDP packets are generally simpler and have lower overhead compared to TCP packets.

Identify Protocols Using TCP

1. Filter Wireshark captures for a specific TCP port, such as HTTP (port 80) or HTTPS (port 443).
2. Analyze the captured packets to understand the protocol's behavior.

View the utput of TCP Traffic captured on UDP Port 80:

*Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Start capturing packets

No.	Time	Source	Destination	Protocol	Length	Info
1072...	2258.716459	23.3.70.11	172.16.218.13	OCSP	943	Response
1072...	2258.764118	172.16.218.13	23.3.70.11	TCP	54	64626 → 80 [ACK] Seq=241 Ack=890 Win=130304 Len=0
1072...	2258.789140	172.16.218.13	142.250.196.3	TCP	66	64627 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1072...	2258.891912	142.250.196.3	172.16.218.13	TCP	66	80 → 64627 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
1072...	2258.892628	172.16.218.13	142.250.196.3	TCP	54	64627 → 80 [ACK] Seq=1 Ack=1 Win=131072 Len=0
1072...	2258.893288	172.16.218.13	142.250.196.3	HTTP	291	GET /s/wr1/ggQ/MFIwUDBOMEwwSjA7BgUrDgMCGGUABBBQwHTUyHgFAXfi3r4z8dPC7p%2BvIYAQUZm
1072...	2258.905708	142.250.196.3	172.16.218.13	TCP	60	80 → 64627 [ACK] Seq=1 Ack=238 Win=65280 Len=0
1072...	2258.933920	142.250.196.3	172.16.218.13	OCSP	779	Response
1072...	2258.984109	172.16.218.13	142.250.196.3	TCP	54	64627 → 80 [ACK] Seq=238 Ack=726 Win=130560 Len=0
1073...	2259.518141	172.16.218.13	152.195.38.76	TCP	66	64632 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1073...	2259.536847	152.195.38.76	172.16.218.13	TCP	66	80 → 64632 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 SACK_PERM WS=512
1073...	2259.536988	172.16.218.13	152.195.38.76	TCP	54	64632 → 80 [ACK] Seq=1 Ack=1 Win=131328 Len=0
1073...	2259.537230	172.16.218.13	152.195.38.76	HTTP	290	GET /MFEwTzBNMEswSTAJBgUrDgMCGGUABBBRzhKfQYsAHQZZDzb8RtQ5PgSjTjQQUpYz%2BMsZrDyZU
1073...	2259.550135	152.195.38.76	172.16.218.13	TCP	60	80 → 64632 [ACK] Seq=1 Ack=237 Win=65024 Len=0
1073...	2259.609101	152.195.38.76	172.16.218.13	OCSP	791	Response
1073...	2259.651062	172.16.218.13	152.195.38.76	TCP	54	64632 → 80 [ACK] Seq=237 Ack=738 Win=130560 Len=0
1076...	2264.878900	172.16.218.13	152.195.38.76	TCP	66	64635 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1076...	2264.894305	152.195.38.76	172.16.218.13	TCP	66	80 → 64635 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 SACK_PERM WS=512
1076...	2264.894710	172.16.218.13	152.195.38.76	TCP	54	64635 → 80 [ACK] Seq=1 Ack=1 Win=131328 Len=0
1076...	2264.896890	172.16.218.13	152.195.38.76	HTTP	260	GET /pca3.cr1 HTTP/1.1
1077...	2264.947835	152.195.38.76	172.16.218.13	TCP	60	80 → 64635 [ACK] Seq=1 Ack=207 Win=65024 Len=0
1077...	2264.947835	152.195.38.76	172.16.218.13	TCP	1514	80 → 64635 [ACK] Seq=1 Ack=207 Win=67072 Len=1460 [TCP segment of a reassembled
1077...	2264.997630	172.16.218.13	152.195.38.76	TCP	54	64635 → 80 [ACK] Seq=207 Ack=1461 Win=131328 Len=0
1077...	2265.043299	152.195.38.76	172.16.218.13	PKIX-C...	441	Certificate Revocation List
1077...	2265.092542	172.16.218.13	152.195.38.76	TCP	54	64635 → 80 [ACK] Seq=207 Ack=1848 Win=130816 Len=0

Analysis summary

When analyzing HTTP traffic in Wireshark, start by observing the TCP handshake, where the client and server exchange SYN, SYN-ACK, and ACK packets to establish a connection.

Next, the client sends an HTTP request, specifying the action, such as GET or POST, the URL, and relevant headers.

The server then responds with an HTTP response packet that includes a status code, headers, and the requested content. This sequence highlights the structured and reliable nature of HTTP communication over TCP.

Exercises

Exercise 1: Capture and Analyze HTTPS Traffic (TCP)

Objective: Experience how Hypertext Transfer Protocol Secure(HTTPS) operates over TCP by capturing and analyzing its traffic.

1. Open Wireshark using **Run as Administrator** and start a new capture session.
2. In the filter bar, enter `tcp port 443` to capture HTTPS traffic.
3. Open a web browser and visit any secure website (such as `https://www.google.com`).
4. Stop the capture after a few seconds.
5. Analyze the captured packets to identify the TCP three-way handshake (SYN, SYN-ACK, ACK).
6. Observe the encrypted HTTPS traffic and note the absence of visible content due to encryption, while still understanding the flow of requests and responses.

Exercise 2: Capture and Analyze DNS Traffic (UDP)

Objective: Experience how User Datagram Protocol (UDP) operates by capturing and analyzing its traffic.

1. Open Wireshark using **Run as Administrator** and start a new capture session.
2. In the filter bar, enter `udp port 53` to capture UDP traffic.
3. To generate DNS traffic, type `nslookup` command in command prompt.
4. Stop the capture after a few seconds.
5. Analyze the captured data to look for DNS requests and responses. Examine the UDP headers.

Exercise 3: Capture and Analyze ARP Traffic (Layer 2)

Objective: Experience how Address Resolution Protocol (ARP) operates by capturing and analyzing its traffic.

1. Open Wireshark using **Run as Administrator** and start a new capture session.
In the filter bar, enter `ARP` to capture ARP traffic.
2. To generate ARP traffic, type `arp -a` command in the command prompt.
3. Stop the capture after a few seconds.
4. Analyze the capture looking for ARP requests and responses. Examine ARP packet details.

Conclusion

This lab provided a hands-on exploration of both TCP and UDP protocols using Wireshark, allowing you to capture and analyze real-world network traffic. By examining HTTPS traffic over TCP, you observed the structured and reliable communication process, including the critical TCP handshake. In contrast, the analysis of SNMP traffic over UDP highlighted the protocol's simplicity and efficiency in connectionless communication. Through these exercises, you gained a deeper understanding of how different protocols leverage the underlying transport layers to meet specific communication needs, reinforcing the fundamental concepts of network traffic analysis.

Author(s)

Dr. Manish Kumar

Other Contributor(s)

Patsy Kravitz



Skills Network